

Bayesian Analysis of Event Frequency: Assignment 1

Álvaro Méndez Rodríguez de Tembleque^a

^a*Università degli Studi di Padova, Department of physics and astronomy “Galileo Galilei”, City, 12345*

Abstract

In this article, we explore the mathematical and physical framework for analyzing the Cosmic Microwave Background (CMB) angular correlation function. We demonstrate how this function and its derived statistics can be used to test the standard Λ CDM cosmological model, potentially revealing new tensions, particularly concerning the assumption of a flat geometry.

1. Introduction

The goal of this text is to explain what I have done in order to make sure the statistical comparisons presented below are reproducible, robust and clearly traceable to the data and code used. This manuscript documents the computational pipeline, the choices made for data processing, and the statistical tests performed to assess the consistency between observed Planck multipoles and theoretical expectations.

The work aims to (1) describe the correlation-function based statistics used to probe large-angle anomalies in the cosmic microwave background (CMB), (2) present a reproducible implementation using CAMB, Monte Carlo realizations and Planck-style simulations, and (3) provide clear instructions so readers can reproduce and extend the results using the code in the repository. The emphasis is on transparency: every derived number can be traced to an input file or to a well-documented routine in the functions/ directory.

1.1. Physics background and intuition

This subsection provides the physical context and intuition behind the statistics used in the analysis so a reader unfamiliar with the implementation can understand why the chosen diagnostics are informative.

The cosmic microwave background (CMB) temperature field on the sphere is a realization of a random process that is to very good approximation statistically isotropic and (to leading order) Gaussian. Its statistical information is compactly encoded in the angular power spectrum C_ℓ , defined by the ensemble average of spherical-harmonic coefficients $a_{\ell m}$: $\langle a_{\ell m} a_{\ell' m'}^* \rangle = C_\ell \delta_{\ell\ell'} \delta_{mm'}$. In practice we work with the commonly used scaled quantity $D_\ell = \ell(\ell+1)C_\ell/(2\pi)$ which emphasizes the contribution per logarithmic interval in multipole.

Large-angle anomalies are features visible at low multipoles ($\ell \lesssim 30$) where cosmic variance (the intrinsic sample variance because we observe only one sky) is large. A direct comparison of observed multipoles to theory must therefore account for the full sampling distribution of the estimated multipoles and derived quantities. This motivates the use of Monte Carlo and pipeline simulations that reproduce the estimator behaviour and the effects of masks, beams and noise.

The two-point correlation function $C(\theta)$ is the harmonic-space power spectrum transformed back to real (angular) space by a sum over Legendre polynomials. Because it is a sum over a wide range of ℓ , the correlation function is often more intuitive for discussing angular separations: features at large angles receive contributions from many low- ℓ modes and are therefore sensitive to coherent deviations at the largest scales.

Two complementary statistics are emphasised in this work:

- The interval-averaged correlation $\langle \xi \rangle_a^b$ is a lin-

*Corresponding author

Email address:

alvaro.mendezrodriguezdetembleque@studenti.unipd.it
(Álvaro Méndez Rodríguez de Tembleque)

ear functional of $C(\theta)$ that summarizes the mean correlation within a chosen angular window. Being linear in the data, it is directly sensitive to shifts in the mean correlation and is relatively straightforward to propagate errors for.

- The quadratic statistic S_a^b (including the classic $S_{1/2}$ choice) integrates the square of the correlation over an interval. Because it is quadratic in the map (or in $C(\theta)$), it is sensitive to the variance of correlations and can emphasize deviations that are small in mean but large in dispersion across realizations. The matrix T_{nm} that appears in the analytic expression encodes the angular overlap of Legendre modes within the interval and must be computed accurately for robust error estimates.

An additional simple but useful diagnostic is C_{180} , the antipodal correlation, which probes correlations between widely separated points and is comparatively insensitive to small-scale masking effects.

Why two types of statistics? The linear $\langle \xi \rangle$ is less affected by distributional skew or amplified tails and often provides a stable test for mean shifts, while the quadratic S statistic can highlight broader classes of departures from Gaussian predictions (for example excess variance or coherent real-space structure). Using both statistics in parallel gives a more complete picture of possible anomalies.

Role of simulations and modelling: to assess whether an observed statistic is anomalous one must know its sampling distribution under the null hypothesis (usually the standard cosmological model). There are three practical ways to generate such distributions in the code:

1. Per- ℓ Gaussian sampling (fast): draw independent Gaussian perturbations of each reported D_ℓ using reported per- ℓ errors and compute derived statistics. This approach is useful for quick checks but neglects estimator covariance introduced by masking and beam corrections.
2. Planck-style forward pipeline (realistic): generate full-sky realizations from a theoretical C_ℓ , apply beam and pixel windows, add noise, and run the same estimator as used on the real data. This reproduces measurement biases and mode coupling induced by the mask and is therefore the preferred route when available.
3. MCMC ensembles (theory sampling): when

posterior samples of cosmological parameters are available (Planck chains), converting each posterior sample to a CAMB spectrum and then to D_ℓ produces an ensemble that captures theoretical uncertainty propagated from parameter estimation.

Practical interpretation and caveats: p-values reported in this work are empirical: for each statistic we compare the observed value to the distribution derived from the chosen ensemble and report the fraction of simulations as extreme (two-sided) as the observation. Important caveats include the “look-elsewhere” effect (multiple, correlated statistics increase the chance of finding at least one low p-value by chance) and the strong influence of cosmic variance at low ℓ . Therefore an anomalously low p-value should be interpreted as a prompt for further investigation rather than definitive evidence for new physics.

This subsection aims to equip a reader with the conceptual understanding needed to follow the code mapping: the next section (Methodology) documents how these statistics are implemented and where to find the corresponding routines in the repository.

2. Methodology

This section describes the data sources, the numerical implementation, and the exact sequence of processing steps required to reproduce the reported results. The repository contains a set of scripts and a directory structure (for example `run_{lmax}_{n}`) that collect simulation outputs, theory chains and auxiliary files; the Methods below explain which files are produced and how they are used by the plotting and analysis routines.

We emphasize two practical points that improve reproducibility for users of this repository:

- All Monte Carlo and chain outputs are saved as human-readable or standard binary arrays (e.g. `.npy`, `.txt`) in the run directory; when present the plotting functions will read these files rather than any intermediary pickle files.
- A boolean flag (`use_recomputed_statistics`) is provided to instruct the plotting utilities to use statistics recomputed from a replacement `D_ell` file (the recomputed files live under `run_dir/recomputed_statistics`). By default

this flag is false, matching the principle that analysis should reproduce published results unless a replacement dataset is intentionally provided.

The remainder of this subsection provides details about the specific statistics and the files produced by each stage.

2.1. Functions and statistics

The analysis relies on three related observables computed from a power spectrum D_ℓ (equivalently C_ℓ):

1. The two-point correlation function $C(\theta)$, which probes temperature correlations as a function of angular separation.
2. The interval-averaged correlation statistic $\langle \xi \rangle_a^b$, a linear functional of $C(\theta)$ that summarizes the mean correlation within an angular interval $[\theta_a, \theta_b]$ (expressed below in terms of $a = \cos \theta_b$, $b = \cos \theta_a$ for convenience).
3. The integrated squared-correlation S_a^b (originally $S_{1/2}$ when $a = \cos(60^\circ)$ and $b = \cos(180^\circ)$), which emphasizes power on a given angular range and is quadratic in the spectrum.

Below we summarise each quantity and state how it is computed and stored in the repository outputs.

Correlation function. The correlation function is defined as

$$C(\theta) = \sum_{\ell} \frac{2\ell + 1}{2\ell(\ell + 1)} D_{\ell} P_{\ell}(\cos \theta)$$

where $D_{\ell} = \ell(\ell + 1)C_{\ell}/(2\pi)$ in the conventions used here. In practice the summation runs from $\ell = 2$ to a configured $\ell_{\max}$ (the code parameter `lmax`). The implementation used to compute $C(\theta)$ is provided in Appendix A, Listing 1; the function supports both direct use of a CAMB-produced theoretical spectrum and Monte Carlo realizations produced by the Planck-style pipeline.

Computed outputs and storage:

- Per-realization correlation arrays are saved as `Corr.npy` in the simulation or model directory.
- The mean correlation used for plotting and comparison is written as the mean across realizations and also exposed via the plotting helpers (see `functions/plots.py`).

Interval-averaged correlation $\langle \xi \rangle_a^b$. The interval average is defined by

$$\langle \xi \rangle_a^b = \frac{\int_a^b C(\theta) d \cos \theta}{b - a} = \sum_{\ell} \frac{D_{\ell}}{2\ell(\ell + 1)} \frac{1}{b - a} [P_{\ell+1}(\cos \theta) - P_{\ell-1}(\cos \theta)]$$

This is implemented in the repository (see Appendix A, Listing 2) and the per-realization values are saved as columns in `xiv_statistic.txt` (with a corresponding `columns.txt` describing the order).

C_{180} and S_a^b . The C_{180} statistic is simply $C(180^\circ)$, i.e. the antipodal correlation; it is robust to certain mask effects because it samples the largest angular separation. The quadratic statistic is defined as

$$S_a^b = \int_a^b C(\theta)^2 \sin \theta d\theta = \sum_{n,m} A_n A_m D_n D_m T_{nm}$$

where T_{nm} are overlap integrals of Legendre polynomials over the interval. The code computes the T_{nm} matrix analytically (see Appendix A, Listing 3) and writes per-realization S values to `S_statistics.txt` and a separate file `C180_statistics.txt` for the antipodal values.

Implementation notes:

- The code carefully handles edge cases where single-realization outputs reduce to 1D arrays and ensures consistent column ordering via `columns.txt` files.
- All text outputs accompanying a run are plain ASCII tables (`.txt`) so they can be reloaded by other tools without requiring the Python source.

2.2. Experimental values

The experimental inputs used throughout the analysis are taken from the Planck legacy archive and the associated likelihood releases compiled by the Planck Collaboration. Concretely, the quantities used are the binned and deconvolved temperature multipoles D_{ℓ} (or equivalently C_{ℓ}) and their reported uncertainties. When reproducing these results the following practical choices were applied:

- **Source and versioning:** the specific Planck chains and central best-fit products are documented in the repository’s `README.md` and were downloaded from the Planck Legacy Archive. Users should verify the exact file timestamps if precise bitwise reproducibility is required.
- **Multipole selection:** for large-angle statistics the low multipoles ($\ell \lesssim 30$) carry most of the signal; however the pipeline accepts any configured ℓ_{max} and the scripts record the value used in the run directory.
- **Uncertainties and mask treatment:** the Planck collaboration provides covariance or per- ℓ error estimates depending on the data product. In this work we treat reported multipole errors as Gaussian 1σ uncertainties when generating naive per- ℓ Monte Carlo samples (see Appendix A). For more realistic simulations the Planck-style pipeline (`functions/simulation.py`) propagates mask, beam and noise effects explicitly when `run_pipeline=True`.
- **Reproducible ingest:** the function `Data_loader.experimental_values(...)` in the codebase centralizes the parsing of experimental inputs and returns a dictionary of named statistics (e.g. `s12_60_180` or `xiv_...`) that the plotting routines use. This ensures consistent ordering and naming between experimental and simulated columns.

2.3. Theory and simulations

2.3.1. Theory

Theoretical predictions are generated with CAMB. The code calls CAMB with a vector of cosmological parameters sampled either from a best-fit vector or from an MCMC chain, then extracts the temperature power spectrum and converts it to the D_ℓ convention used throughout the repository. The CAMB calls are implemented in `functions/cosmo.py` (see Listing 1).

Parameters: the full set of sampled parameters is listed below and matches the labels used in the Planck chains. When a chain is available the code reads the last N samples (by default 1000) to form an ensemble of theoretical spectra; when only a best-fit entry is used the code instead samples per- ℓ Monte Carlo realizations around the reported D_ℓ (see below).

- `ombh2`, `omch2`, `H0`, `omk`, `YHe`, `nnu`, `nrun`, `Alens`, `ns`, `As`, `w`, `wa`, `mnu`, `tau`

2.3.2. Simulation pipelines

Two complementary strategies are provided to obtain ensembles of D_ℓ and derived correlation statistics:

1. **Per- ℓ Monte Carlo (fast, approximate).** Starting from a vector of reported multipoles and their 1σ errors, the code draws independent Gaussian realizations for each ℓ (or uses the per- ℓ sampling that simulates the underlying $a_{\ell m}$ modes when available). This method is computationally cheap and useful to estimate sampling distributions under the simplifying assumption of independent multipoles; its implementation appears in Listing 2 and the results are written to `simulation/D_ell.npy`, `simulation/Corr.npy`, and the scalar statistic files (`S_statistics.txt`, `xiv_statistic.txt`, `C180_statistics.txt`) together with `columns.txt`.
2. **Planck-style pipeline (detailed, realistic).** The code can instead run a Planck-like simulation that generates full-sky realizations of the CMB given a theoretical C_ℓ , applies the instrumental beam and pixel window, adds a noise model, and estimates D_ℓ from the simulated sky maps using the same estimator as in the Planck pipeline. This yields realistic covariance between multipoles induced by masking, beam convolution and noise. The parallel implementation is available in `functions/simulation.py` (Listing 3) and stores outputs in the same directory structure as above for downstream analysis.

MCMC-based ensembles. When a parameter chain (getdist or plain text chains) is available the script `chain_results(...)` (in `functions/cosmo.py`) fetches the requested final segments of the chain, converts each sample to CAMB cosmological parameters, computes the theoretical spectrum and correlation for each sample, and stores the per-sample arrays under `theory/<model>/D_ell.npy` and `theory/<model>/Corr.npy`. The corresponding scalar statistics for each sample are written to `theory/<model>/S_statistics.txt`, `xiv_statistic.txt` and `C180_statistics.txt` and a `columns.txt` file is included to guarantee consistent ordering.

Run directory layout and bookkeeping. Each analysis run is organized under a directory

named `run_{lmax}_{n}` where the simulation outputs and theory models are stored in the subfolders `simulation/` and `theory/<model>/`. When the optional recompute step (see `recompute_statistics` in `main.py`) is invoked it writes replacement statistic files to `run_.../recomputed_statistics/`; the plotting utilities expose a flag `use_recomputed_statistics` that directs them to read these recomputed files if desired. This separation keeps original run outputs intact while allowing straightforward replacement for dedicated tests.

Outputs and reproducibility notes.

- All numeric arrays are saved as `.npy` where possible (fast binary reload) and scalar tables as ASCII `.txt` to ease inspection.
- The repository scripts record key metadata (`lmax`, number of realizations, random seeds when used) inside the run folder so reported figures can be traced back to the exact inputs.

Following the generation of theoretical and simulated ensembles, the pipeline computes the statistics described above for every realization and uses those distributions to assign empirical p-values when comparing to the experimental measurements.

3. Results

Below is a brief description of the cosmological models analysed in the figures that follow. The `base_plikHM_TT` and `base_CamSpecHM_TT` entries are the standard six-parameter Λ CDM fits using the Plik and CamSpec high-multipole TT likelihoods respectively, while the `base_omegak_*` variants additionally allow spatial curvature (Ω_k) to vary. Panels labelled “best-fit” use a single best-fit theoretical spectrum with per- ℓ Gaussian Monte Carlo realizations; panels labelled “MCMC” show ensembles generated from posterior samples.

3.1. MCMC sampling results

This subsection presents the empirical distributions obtained from the MCMC-based ensemble. Figures below show histograms of the simulated statistics (simulation-derived distributions) together with the experimental value (vertical marker). The images were produced by the repository plotting utilities and saved under the example run directory `run_30_100/images/histograms`.

Notably, when the ensemble is constructed from the best 1000 MCMC posterior samples the observed experimental statistics tend to lie in the tails of the MCMC-derived distributions, indicating that the data is not well described by that sampled-theory ensemble. In contrast, using the single best-fit spectrum and generating per- ℓ Monte Carlo realizations around that best-fit produces empirical distributions that are consistent with theoretical expectations: the observed statistics fall within the simulated distributions. This distinction suggests the apparent tension depends on the choice of ensemble (MCMC sampling versus best-fit Monte Carlo) rather than signaling an outright breakdown of the underlying theoretical model; possible origins and caveats are discussed below.

3.1.1. Model: `base_plikHM_TT`

MCMC-derived histograms for model `base_plikHM_TT` (from `run_30_100`).

3.1.2. Model: `base_CamSpecHM_TT`

MCMC-derived histograms for model `base_CamSpecHM_TT` (from `run_30_100`).

3.1.3. Model: `base_omegak_CamSpecHM_TT`

MCMC-derived histograms for model `base_omegak_CamSpecHM_TT` (from `run_30_100`).

3.1.4. Model: `base_omegak_plikHM_TTTEEE`

MCMC-derived histograms for model `base_omegak_plikHM_TTTEEE` (from `run_30_100`).

3.2. Best-fit (per- ℓ) results

The panels below show the histograms obtained using the best-fit power spectrum and per- ℓ Monte Carlo realizations (the “best-fit” mode). These images are stored under `run_30_10001/images/histograms` and demonstrate the alternative (fast) sampling approach where per- ℓ Gaussian draws are used instead of full pipeline realizations.

3.2.1. Model: `base_plikHM_TT (best-fit)`

Best-fit per- ℓ histograms for model `base_plikHM_TT` (from `run_30_10001`).

3.2.2. Model: `base_CamSpecHM_TT (best-fit)`

Best-fit per- ℓ histograms for model `base_CamSpecHM_TT` (from `run_30_10001`).

3.2.3. Model: *base_omegak_CamSpecHM_TT*
(best-fit)

Best-fit per- ℓ histograms for model
base_omegak_CamSpecHM_TT (from
run_30_10001).

3.2.4. Model: *base_omegak_plikHM_TTTEEE*
(best-fit)

Best-fit per- ℓ histograms for model
base_omegak_plikHM_TTTEEE (from
run_30_10001).

4. Appendix A: Code and implementation

Below are important implementation excerpts used in this work. Full source code and additional helper functions are available in the `functions/` directory. These snippets illustrate the generation of Monte Carlo realizations, the computation of CAMB spectra and correlations, and the Planck-style simulation pipeline used to produce observed-like D_l realizations.

```
#| label: lst-genDl_cosmo
#| caption: "Generation of per-ell D_l
  ↳ realizations (functions/cosmo.py)"

def generate_Dl_realizations(row, n):
    """
    Generate n realizations for a given
    ↳ row. If the provided D_ell is an array
    ↳ (full spectrum)
    use the parallel planck-style
    ↳ pipeline, otherwise fall back to the
    ↳ original per-ell sampling.
    """
    D_ell = row['L']

    # == original per-ell Monte Carlo
    ↳ sampling (fallback) ==
    ell = int(row['#'])
    D_ell_scalar = float(D_ell)

    F = ell * (ell + 1) / (2 * np.pi)
    C_ell = D_ell_scalar / F #
    ↳ convert back to C_ell

    # Protect against negative or NaN
    ↳ C_ell which would make the
    ↳ variance invalid for sampling
    safe_C = C_ell if (np.isfinite(C_ell)
    ↳ and C_ell > 0.0) else 0.0
```

```
results = []

for _ in range(n):
    # Storage for a_lm from
    ↳ m=-ell..+ell
    a_lm = np.zeros(2*ell+1,
    ↳ dtype=np.complex128)

    # m = 0 → purely real Gaussian
    a_lm[ell] = np.random.normal(0,
    ↳ np.sqrt(safe_C))

    # m > 0
    for m in range(1, ell+1):
        re = np.random.normal(0,
        ↳ np.sqrt(safe_C/2))
        im = np.random.normal(0,
        ↳ np.sqrt(safe_C/2))
        a_lm[ell + m] = re + 1j*im

    # m < 0 via symmetry
    a_lm[ell - m] = (-1)**m *
    ↳ np.conj(a_lm[ell + m])

    # Compute C_ell estimator
    Cl_hat = np.sum(np.abs(a_lm)**2) /
    ↳ (2*ell + 1)

    # Convert to D_ell
    Dl_hat = F * Cl_hat

    results.append(Dl_hat)

return results
```

```
#| label: lst-compute_cl_cor_pl
#| caption: "Computing CAMB D_l and
  ↳ correlation (functions/cosmo.py)"

def compute_cl_cor_pl(parss, lmax, xvals,
    ↳ run_pipeline=False,
    ↳ nside_in=2048,
    ↳ fwhm_in_arcmin=40.0, noise_Cl=None,
    ↳ mask=None,
    ↳ num_workers=1):
    try:
        pars =
        ↳ camb.set_params(ombh2=parss['omegab2'],
        ↳ omch2=parss['omegach2'],
        ↳ H0=parss['H0'], omk=parss['omegak'],
```

```

↪ YHe=parss['yheused'],
↪ nnu=parss['nnu'], nrun=parss['nrun'],
↪ Alens=parss['Alens'],

↪ ns=parss['ns'],
↪ As=np.exp(parss['logA']) * 1e-10,
↪ w=float(-1), wa=parss['wa'],

↪ mnu=parss['mnu'], tau=parss['tau'])
except TypeError as e:
    logger.exception(f"TypeError in
↪ compute_cl_cor_pl: {e}")
    for key, value in parss.items():
        logger.debug(f" Parameter
↪ {key}: {value} (type: {type(value)})")
        raise

    resu = camb.get_results(pars)
    cls = resu.get_cmb_power_spectra(pars,
↪ CMB_unit='muK', lmax=lmax)
    totCL = cls['total']

    # theoretical D_l from CAMB (starts at
    ↪ l=2)
    TTCl_theory = totCL[2:, 0]

    if run_pipeline:
        try:
            ell = np.arange(2, lmax + 1,
↪ dtype=int)
            df_sim =
↪ planck_style_pipeline_parallel(
                ell=ell,

↪ D_ell_theory=[TTCl_theory],
                nside_in=nside_in,

↪ fwhm_in_arcmin=float(fwhm_in_arcmin),
                noise_Cl=noise_Cl,
                mask=mask,
                estimate_Cl=True,
                num_workers=num_workers,
            )
            recovered_D_ell =
↪ np.asarray(df_sim['D_ell_arr'].iloc[0])
        except Exception as e:
            logger.exception(f"Planck
↪ pipeline failed in compute_cl_cor_pl,
            ↪ falling back to theory: {e}")
            recovered_D_ell = TTCl_theory

```

```

else:
    recovered_D_ell = TTCl_theory

    # convert 1D recovered D_l (starts at
    ↪ l=2) to full cls array for
    ↪ camb.cl2corr
    ncomp = totCL.shape[1]
    full_cl = np.zeros((lmax + 1, ncomp),
↪ dtype=float)
    ell_idx = np.arange(2, lmax + 1,
↪ dtype=int)
    full_cl[ell_idx, 0] = recovered_D_ell

    totCorr_rec =
↪ camb.correlations.cl2corr(full_cl,
    ↪ xvals, lmax)
    TTCor_rec = totCorr_rec[:, 0]

    return recovered_D_ell, TTCor_rec,
    ↪ TTCor_rec[-1]

```

```

#| label: lst-planck_pipeline
#| caption: "Planck-style parallel
↪ simulation pipeline
↪ (functions/simulation.py)"

def planck_style_pipeline_parallel(
    ell, D_ell_theory, nside_in,
    ↪ fwhm_in_arcmin, noise_Cl,
    mask=None, estimate_Cl=True,
    ↪ num_workers=None
):
    ell = np.asarray(ell, dtype=int)
    if isinstance(D_ell_theory,
    ↪ (pd.Series, list, tuple)):
        D_ell_theory =
↪ np.asarray(list(D_ell_theory),
        ↪ dtype=float)
    else:
        D_ell_theory =
↪ np.asarray(D_ell_theory, dtype=float)
    n_sims = D_ell_theory.shape[0]

    # Convert D_ell to C_ell for all rows
    ↪ at once
    F = ell * (ell + 1) / (2 * np.pi)
    C_ell = D_ell_theory / F

    # Beam and pixel window
    sigma = np.radians(fwhm_in_arcmin /
    ↪ 60) / np.sqrt(8 * np.log(2))

```

```

b_ell = np.exp(-0.5 * ell * (ell + 1)
↪ * sigma**2)
p_ell = hp.pixwin(nside_in,
↪ lmax=int(np.max(ell))) [ell]

Cl_to_simulate = C_ell * (b_ell**2) *
↪ (p_ell**2)

if num_workers is None:
    num_workers = max(1,
↪ int(cpu_count() - 4))

tasks = [
    (Cl_to_simulate[i], ell, nside_in,
↪ fwhm_in_arcmin, noise_Cl, mask,
↪ estimate_Cl)
    for i in range(n_sims)
]

with Pool(processes=num_workers) as
↪ pool:
    results =
↪ pool.map(_single_realization_worker,
↪ tasks)

Cl_matrix = np.array(results)

# Convert back to D_ell
D_ell_out = Cl_matrix * F
df_out = pd.DataFrame({'D_ell_arr':
↪ [D_ell_out[i] for i in
↪ range(n_sims)]})
return df_out

```

These snippets are cited from the repository and are referenced in the main text (Methodology). Full source and additional helper functions are available in `functions/cosmo.py` and `functions/simulation.py`.

5. Appendix B: Error propagation

The error propagation assumes that the multipoles are independent of each other and have a Gaussian distribution. Also, it is assumed that the angle has no error at all.

5.1. Error of the correlation function

The error of the correlation function can be calculated according to this derivation

$$\Delta^2 C(\theta) = \left(\frac{\partial C(\theta)}{\partial D_l} \Delta D_l \right)^2 = \left(\sum_l \Delta D_l \frac{2l+1}{2l(l+1)} P_l(\cos(\theta)) \right)^2$$

$$\Delta C(\theta) = \sqrt{\left(\sum_l \Delta D_l \frac{2l+1}{2l(l+1)} P_l(\cos(\theta)) \right)^2}$$

with this function

```

#| label: 1st-correlation_func_err
#| caption: "Code to calculate the error
↪ of the correlation function
↪ (functions/correlation_function.py)"

def correlation_func_err(error,xvals):
    cor_err=0

    fac_err=
    ↪ [(2*(l+2)+1)/(2*(l+2)*((l+2)+1))*c for
    ↪ l,c in enumerate(error)]
    for l,f in enumerate(fac_err):
        cor_err+=(f*P(l+2,xvals))**2
    return cor_err**0.5

```

5.2. Error of $S_{1/2}$

$$\Delta^2 S_a^b = \left(\frac{\partial S_a^b}{\partial D_n} \Delta D_n \right)^2 + \left(\frac{\partial S_a^b}{\partial D_m} \Delta D_m \right)^2$$

$$= \left(\sum_n \sum_m \frac{2n+1}{2n(n+1)} \frac{2m+1}{2m(m+1)} D_m T_{nm} \Delta D_n \right)^2 + \left(\sum_n \sum_m \frac{2n+1}{2n(n+1)} \frac{2m+1}{2m(m+1)} D_n T_{nm} \Delta D_m \right)^2$$

$$= \sum_n \sum_m \left(\frac{2n+1}{2n(n+1)} \frac{2m+1}{2m(m+1)} T_{nm} \right)^2 \left(D_m^2 \Delta D_n^2 + D_n^2 \Delta D_m^2 \right)$$

$$\Delta S_a^b = \sqrt{\sum_n \sum_m \left(\frac{2n+1}{2n(n+1)} \frac{2m+1}{2m(m+1)} T_{nm} \right)^2 \left(D_m^2 \Delta D_n^2 + D_n^2 \Delta D_m^2 \right)}$$

with this function

```

#| label: 1st-S12_err
#| caption: "Code to calculate the error
↪ of the old statistic
↪ (functions/s12.py)"

```

```

def S12_err(D_ell,D_ell_err,M):
    # Set the precision high enough to
    ↪ handle large calculations
    getcontext().prec = 1000
    s=Decimal(0)
    for i,xn in enumerate(D_ell_err):
        n=i+2
        fac1=((2*n+1))/(2*n*(n+1))
        for j,xm in enumerate(D_ell):
            m=j+2
            fac2=((2*m+1))/(2*m*(m+1))
            Amn=fac1*fac2
            integral=M[i,j]

            s +=
    ↪ Decimal(Amn**2)*Decimal(integral**2)*Decimal(D_ell[i]**2*xm**2
    ↪ + D_ell[j]**2*xn**2)

    return float(s)**0.5

```

5.3. Error of $\langle \xi \rangle_a^b$

```

#| label: lst-xivar_err
#| caption: "Code to calculate the error
↪ of the new statistic
↪ (functions/xiv.py)"

def xivar_err(D_ell_err, a,b):
    s=0
    for i,d in enumerate(D_ell_err):
        l=i+2
        fac=d/(2*l*(l+1))

        integral = Decimal(legendre(l + 1,
    ↪ b) - legendre(l - 1, b)-legendre(l +
    ↪ 1, a) + legendre(l - 1, a))

        s+=(fac*float(integral)/(b-a))**2

    return (s)**0.5

```